Optimización y Comparación de Métodos en Simulaciones de N-Cuerpos: Barnes-Hut vs. Métodos Directos

Departamento de Física, Facultad de Ciencias, Universidad Nacional de Colombia, Bogotá D.C, Colombia

Diaz Chiquillo, Braither. Ortega Perenguez, Edison. Vargas Rodriguez, Sergio. Victorino Rojas, Juan
brdiazc@unal.edu.co, eortegap@unal.edu.co, svargasro@unal.edu.co, jvictorino@unal.edu.co

1. Introducción.

La cosmología actual, desde el modelo ΛCDM, indica que la materia visible solo contribuye con el 5 % de la gravedad que puede medirse. El restante 95 %, dividido en un 26 % de materia oscura y un 69 % de energía oscura, constituye las componentes predominantes según esta descripción.

Esta situación ha llevado a proponer varios candidatos teóricos que prometen explicar la naturaleza de la materia y la energía oscura. En [1], se explora la posibilidad de un universo en el que existe y se crea masa negativa. Dicha idea se basa en la búsqueda de una solución que preserve la simetría en el universo, haciendo de la polarización una propiedad fundamental del mismo, observada en diversos fenómenos, como lo son la carga eléctrica la cual tiene carga positiva y negativa, la carga magnética cuya fuente es un dipolo, o la información cuántica dada en 0 o 1.

Aunque la masa negativa no aparenta ser una idea intuitiva, pues gran parte de las teorías sobre la gravedad se han desarrollado siempre tomando una carga gravitacional (masa) del mismo signo, el estudio de sus comportamientos en relación con la física conocida, evidencia, teóricamente, la capacidad de producir efectos muy similares a la materia y energía oscura a la vez que comparte propiedades esperadas como la generación de efectos repulsivos, la imposibilidad de formar estructuras como estrellas o producir la expansión del universo. Así, el modelo propuesto puede ser una forma de unificar en un único marco teórico a la energía y materia oscura.

De esta manera, el autor propone dos enfoques para verificar la hipótesis. Primero, se aborda desde un aspecto teórico, para luego centrarse en una de las estrategias más eficaces en la investigación de interacciones entre partículas puntuales: la simulación de N cuerpos. Es en este último enfoque se centrará el presente trabajo. El código presentado en [1] está implementado en Python y utiliza las bibliotecas NumPy, Matplotlib y Dask. La razón principal detrás de esta elección, según la perspectiva del autor, no es la optimización de rendimiento, sino la facilidad y comprensibilidad del código. Por esta razón, se emplean métodos más simples en comparación con los utilizados en enfoques más vigentes, como los expuestos en [2]. Cabe resaltar que el estudio se limita al primer resultado obtenido por Farnes en [1].

[1] Utilizar la segunda ley de Newton para calcular el movimiento de los cuerpos es una aproximación adecuada para velocidades pequeñas en comparación con la velocidad de la luz.

Lo anterior es lo que motiva el desarrollo de nuestra propuesta de trabajo, cuyo propósito es implementar el algoritmo de Barnes-Hut en la simulación de N cuerpos propuesta en [1], utilizando Python. La meta principal es comparar el tiempo de ejecución de ambos métodos, centrándose especialmente en la eficiencia del algoritmo de Barnes-Hut dentro del contexto del modelo de masa negativa y analizando cómo optimiza los tiempos de ejecución.

1.1. Simulación N cuerpos.

Como fue mencionado, las simulaciones de N cuerpos son una herramienta fundamental en la investigación de interacciones entre partículas puntuales. Una simulación de N cuerpos consiste, en esencia, en resolver numéricamente las ecuaciones de movimiento para una partícula sometida a una cierta fuerza a partir de la segunda ley de Newton.[1] De esta forma, conociendo la fuerza o interacción, es posible actualizar la aceleración y, por medio de técnicas de integración, calcular la velocidad y posición. Existen diversos métodos para resolver las ecuaciones de movimiento. Para este proyecto serán utilizados y analizados el método directo y también el método "Verlet integration".

El método directo consiste en resolver las ecuaciones de movimiento de forma directa a partir de la fuerza. Por ejemplo, si se tiene un sistema de N partículas con masa m_i en el espacio y se considera únicamente la fuerza gravitacional, en el instante de tiempo $t = 0$ la fuerza que siente la i-esima partícula debido a la j-esima partícula viene dada por:

$$\vec{F}_{ij} = -G\frac{m_i m_j}{r^2}\hat{r}_{ij} \tag{1}$$

Donde $\vec{r} = \vec{r}_i - \vec{r}_j$ es el vector que une a ambas partículas. En este caso, cada iteración del programa es un paso en el tiempo $\Delta t = 1$ y en cada instante de tiempo la aceleración toma una constante como resultado de la fuerza en ese mismo instante, por lo que la posición y velocidad se pueden determinar como se observa en la ecuación 2.

$$\begin{aligned} \mathbf{x}(t+\Delta t) &= \mathbf{x}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2}\mathbf{a}(t)\Delta t^2 \\ \mathbf{v}(t+\Delta t) &= \mathbf{v}(t) + \mathbf{a}\Delta t \end{aligned} \tag{2}$$

Por otro lado, uno de los métodos más utilizados en astrofísica, cosmología y dinámica molecular es "Verlet integration"debido a la sencillez en la implementación, la rever-

sibilidad temporal del algoritmo y la preservación de la forma simpléctica en el espacio de fases. La forma recursiva que adquiere la posición y la velocidad se pueden observar en las ecuaciones 3 y 4.

$$\mathbf{x}(t+\Delta t) = \mathbf{x}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2}\mathbf{a}(t)\Delta t^2 \tag{3}$$

$$\mathbf{v}(t+\Delta t) = \mathbf{v}(t) + \frac{\mathbf{a}(t) + \mathbf{a}(t+\Delta t)}{2}\Delta t \tag{4}$$

2. Implementación.

Ahora, se presenta a continuación el algoritmo de Barnes-Hut que será implementado.

2.1. Algoritmo Barnes-Hut.

El algoritmo de Barnes-Hut, utiliza aproximaciones para el cálculo de las interacciones de largo alcance con el objetivo de simplificar el número de interacciones a una escala de $N * log(N)$, lo cual disminuye considerablemente el tiempo de computación [2].

La modificación con respecto al método directo, ocurre en el ciclo que calcula la fuerza, por lo que la actualización, por medio de integración numérica, no sufre cambio alguno.

El método se fundamenta en la representación jerárquica,de allí el nombre de treecodes, del espacio geométrico que almacena los cuerpos que son objeto de estudio. Se introduce la idea clave de la descripción de un ensamble de cuerpos y su distribución de masa a través de un pseudo-cuerpo, de esa forma se pueden reemplazar las interacciones de varias partículas con la de un solo gran cuerpo. Se ha de tener en cuenta que la aproximación solo es válida cuando se tiene una interacción lo suficientemente lejos, lo cual es controlado por parámetros definidos en el método. Veamos entonces, cuándo es válida la aproximación y cuál es el algoritmo para su construcción.

Se describe el algoritmo para dos dimensiones, puesto que es más sencillo de visualizar**.**

Primero, se crea la raíz del árbol -Quadtree[2]-, que representa las fronteras de la caja que contiene a todas las partículas. Luego, la raíz se divide en cuatro cajas iguales dividiendo cada línea de la frontera por la mitad, generando así cuatro hijos, nodos o celdas, los cuales abarcan $\frac{1}{4}$ del área de su padre. Este proceso de subdivisión continúa de manera recursiva hasta que cada nodo contiene exactamente una partícula, momento en el cual se le denomina nodo puntual o hoja. Esta descripción se ilustra en la figura 1.

Cabe aclarar que el algoritmo del paso anterior puede ser modificado. La modificación es la utilizada para el presente informe. En vez de meter todas las partículas dentro del árbol y luego recursivamente dividir las celdas en nodos puntuales, el árbol se construye una partícula a la vez. Es decir, se dividen las celdas y se insertan las partículas una a una, en niveles inferiores de los nodos -subnodos- cuando esto sea necesario, con el fin de conseguir que todas las partículas terminen almacenadas en un nodo puntual.

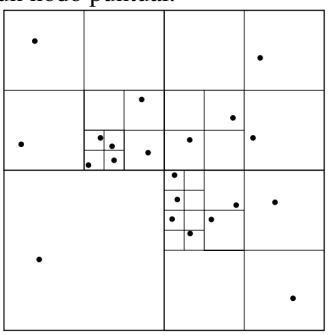


Figura 1. Distribución de 21 partículas contenidas en un Quadtree. Tomado de [2].

Una vez se tienen solo nodos puntuales se hace la representación de cada nodo en un pseudo-cuerpo, desde el nodo más pequeño al más grande, ubicando el centro de masa de cada uno de los nodos y grupos de nodos. Además, se le asigna a cada uno el atributo de masa total, el cual es la suma de las masas de sus componentes.

Así, una vez descompuestos los pseudo-cuerpos, se puede calcular la fuerza que le hacen a una partícula de prueba. Para ello, es necesario establecer la noción de estar "bien separado", pues como se mencionó, la aproximación solo se puede usar cuando dicho criterio se cumple, en caso contrario se debe calcular la interación partícula-partícula. El parámetro que controla cuándo una partícula de prueba está "bien separada" de un pseudo-cuerpo, se denomina ángulo de apertura ó criterio de aceptación multipolar y se simboliza con θ. Para saber si un nodo está lo suficientemente lejos de una partícula de prueba se calcula el cociente entre la longitud de la caja que contiene al nodo, l, y la distancia, d, que separa a la partícula de prueba y el centro de masa del nodo. De ese modo, si $\frac{l}{d} < \theta$, se considera que ambos objetos están bien separados. Lo anterior se ejemplifica en la figura 2.

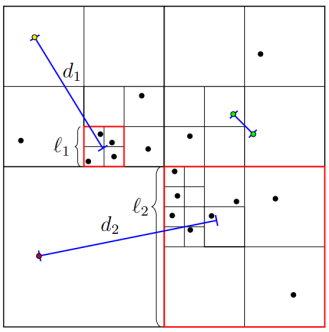


Figura 2. El punto amarillo está bien separado de la celda con longitud lateral l_1 si $\frac{l_1}{d_1} < \theta$. El punto morado está bien separado de la gran celda con longitud lateral l_2 si $\frac{l_2}{d_2} < \theta$. Los extremos de los segmentos de línea en los que no hay una partícula, están en el centro de masa del respectivo conjunto de puntos del nodo. Los dos puntos verdes no están bien separados y su interacción de corto alcance debe calcularse directamente. Tomado de [2].

[2] En 3 dimensiones, al árbol se le llama Octree.

Ahora, para el cálculo de fuerzas sobre una partícula de prueba, se recorre el árbol empezando por los nodos de mayor nivel hacia los de menor nivel, lo cual se implementa recursivamente empezando en el nodo raíz. Si el criterio de aproximación multipolar entre una partícula de prueba y un nodo es aceptable, entonces la interacción de la partícula por dicha rama se completa. En caso de que el criterio no se cumpla, se abre el nodo y se vuelve a evaluar el criterio. Si se da el caso de no cumplirse el criterio y tener que abrir el nodo hasta llegar a un nodo puntual, entonces se calcula la interacción partícula-partícula, tal como en el método directo [3]. Finalmente, completadas todo el cálculo de las interacciones de todas las partículas, se actualizan las velocidades y posiciones de cada partícula.

2.2. Detalles implementación.

Para la implementación del algoritmo de Barnes-Hut en la simulación propuesta por [1], es conveniente aclarar algunos detalles importantes del código original.

Para realizar la simulación, se sitúa una galaxia de masa positiva en el centro de una distribución inicial de partículas. La galaxia de masa positiva sigue el modelo de Hernquist con simetría esférica y se inicializa con un radio de escala de 1.0 [1]. Esta distribución de masa positiva está en el centro de un cubo de masa negativa, con una escala tal que tenga propiedades similares a las de la Vía Láctea. Las condiciones iniciales para las masas negativas se definen mediante una distribución uniforme para su posición inicial y una velocidad inicial de 0.

A continuación se describen algunos detalles relevantes para la implementación del algoritmo de Barnes-Hut. Dicho código puede ser visto en detalle en el archivo adjuntos a este escrito, donde todos los cambios realizados al código original se comentan con dos numerales (##).

El algoritmo se construyó utilizando programación orientada a objetos, ya que era la opción más natural para la implementación. Varias de las clases creadas fueron modificadas en gran medida, pero en principio, se obtuvieron de [3].

Se desarrollaron 4 clases principales:

- **Simulation:** Maneja los datos y conexión entre los objetos que son simulados. Se crea un objeto de Simulation en cada iteración. Tiene como atributos 3 constantes, que son θ, el criterio de aceptación multipolar, G, el valor de la constante de gravitación normalizada y epsilon, el "softening factor". Además, contiene una lista de todos las partículas que serán simuladas. En esta clase es donde se crea y almacena el Octree y sus nodos.
- **SimulationBody:** Almacena la información de un cuerpo simulado. En este caso, cada partícula será un objeto de esta clase. Tiene como atributos la simulación a la que pertenece, masa, posición y velocidad. Contiene los métodos para actualizar las posiciones y velocidades de cada objeto, una vez calculada la fuerza.
- **Octree:** Esta clase, calcula y crea el Octree. Se crea uno de estos en cada paso de la simulación. Tiene como atributos, a todas las partículas simuladas y al nodo raíz. Una vez se inicializa, se insertan todas las partículas en los nodos y maneja la creación de subnodos cuando es necesario. Además, en el Octree también están definidos los métodos para el cálculo de las fuerzas de uno o varios nodos a la partícula. Es aquí donde se calcula si una partícula está bien alejada de otro nodo, tal como se explicó. Una vez se ejecuta el método para calcular las fuerzas, se actualizan los atributos de fuerza de todos los objetos de la clase SimulationBody que pertenecen a el objeto Simulation, que será utilizada luego, en la clase SimulationBody para actualizar las posiciones y velocidades.
- **Node:** Almacena los datos de un Octree. Cada objeto nodo, tiene el atributo de subnodos, el cual es un arreglo tridimensional, que indica si el nodo contiene alguna partícula o nodo en alguno de sus subnodos. Existen los métodos para crear subnodos, los cuales se añaden a una lista de los propios nodos del nodo principal, para llevar registro de qué lleva cada nodo en su interior. También hay definido un método que calcula la distribución de masa de cada nodo del Octree, tal como fue mencionado en la sección anterior.

Así, con las objetos de las clases, que se crean al ejecutar la función "update_velocities_bh", se ejecutan los métodos necesarios para que se calculen las nuevas posiciones y velocidades de los objetos simulados. Dicha información se escribe en un archivo que luego será utilizada para la visualización. Por último, se hace énfasis en que los tiempos que serán reportados en la sección siguiente, es el tiempo en que demora en ejecutarse la función "update_velocities_bh".

3. Resultados y discusión.

A continuación se presentan los resultados obtenidos de la comparación entre ambos algoritmos. Para esta comparación se establecieron tres niveles asociados con la cantidad de cuerpos, el nivel bajo con $N = 500$, donde 50 partículas tienen masa positiva y 450 tienen masa negativa. El nivel medio con $N = 2500$, donde 250 partículas tienen masa positiva y 2250 tienen masa negativa. Y el nivel alto con $N = 5000$ donde 500 partículas tienen masa positiva y 4500 tienen ma-

[3] Para el cálculo de interacciones partícula-partícula, se introduce el llamado "softening factor", el cual se utiliza para reducir efectos de encuentros demasiado cercanos que puede llevar a singularidades y comportamientos no esperados, debido a errores numéricos.

sa negativa. Posteriormente, se evalúa cada uno de los niveles descritos para ambas implementaciones: La propuesta en [1] y la propia.

Para el estudio, se establecieron algunos comandos para recopilar datos, que no están incluidos en el código adjunto, de modo que fuera más sencillo evaluar el tiempo promedio de cómputo y la desviación estándar.

Para la evaluación de todas las iteraciones, se utilizó la misma máquina, a la cual se accedió remotamente a través de la plataforma https://mybinder.org/. Se muestran los resultados obtenidos en la figura 3.

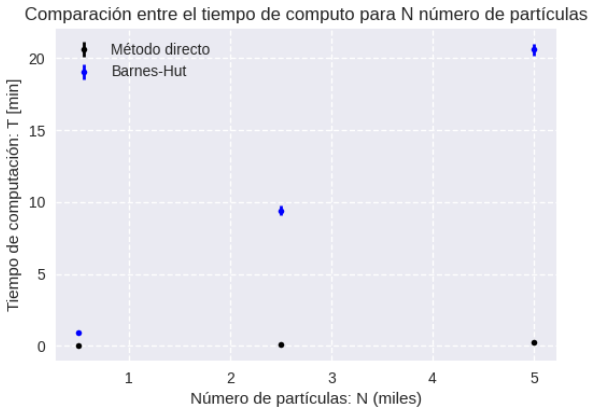


FIGURA 3. Tiempo de ejecución para ambas implementaciones en los niveles mencionados.

Es evidente que el tiempo de ejecución del método de Barnes-Hut no fue inferior al tiempo de ejecución de los métodos directos paralelizados, contrario a lo que esperaba. Lo anterior puede deberse a varias razones.

Primero, si bien el algoritmo de Barnes-Hut es eficiente, puede tener una complejidad agregada al tener la necesidad de construir y navegar por el árbol en cada iteración, especialmente para un conjunto de datos grandes. En este caso, los métodos directos pueden funcionar mejor, pues como son paralelizados, dividen la carga de trabajo en múltiples núcleos de manera explícita.

Otra razón por la cual el algoritmo de Barnes-Hut puede no mostrar la eficiencia esperada podría deberse a la naturaleza inherente del problema. La distribución inicial de las partículas de masa positiva se encuentra localizada en el centro, como se muestra en la figura 4, lo que podría resultar en una reducción insuficiente en el cálculo de las interacciones. Esto conlleva a una pérdida de la paralelización de los procesos, sin que se obtenga una ganancia significativa a cambio.

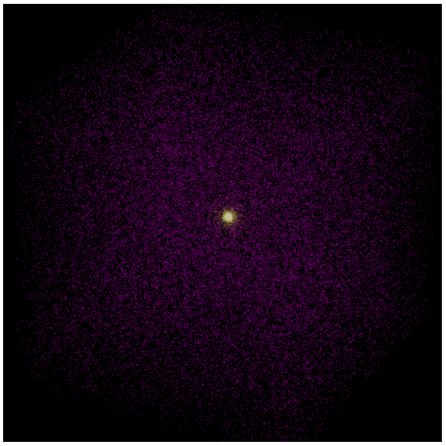


FIGURA 4. Distribución de masa inicial para 50.000 cuerpos.

Se evidencia, por tanto, la importancia de optimizar los algoritmos, y se propone investigar en futuros proyectos la optimización del método de Barnes-Hut, incluyendo la implementación de paralelización en el mismo.

4. Conclusiones.

Aunque el método de Barnes-Hut generalmente supone una mejora en el tiempo de computación en comparación con los métodos directos, en este caso particular, no es evidente. La razón radica en que los métodos directos ejecutados en el código principal incorporan una implementación que paraleliza los procesos, mientras que el algoritmo de Barnes-Hut implementado en este contexto no lo hace. Este hecho resalta la importancia de considerar las características específicas de la implementación y las optimizaciones aplicadas al evaluar y comparar métodos numéricos en un entorno particular. Se planea realizar implementaciones de optimización y paralelización en el modelo de Barnes-Hut en futuros trabajos. Asimismo, se tiene la intención de evaluar su precisión de manera más exhaustiva, ya que en esta ocasión no fue posible debido a las limitaciones de recursos computacionales al intentar incluir un gran número de partículas en el sistema. Estas mejoras y evaluaciones más detalladas contribuirán a una comprensión más completa de las capacidades y limitaciones del método de Barnes-Hut en el contexto específico de la simulación propuesta.

5. Referencias:

1. J. S. Farnes, A unifying theory of dark energy and dark matter: Negative masses and matter creation within a modified CDM framework, Astronomy Astrophysics 620 (2018), https://doi.org/10.1051/0004-6361/201832898
2. A. Brandt, On Distributed Gravitational N-Body Simulations (2022).
3. PhySimdev, Python Gravity Simulation, https://github.com/PhySimdev/Python-Gravity-Simulation (2023).
4. K. Dolag, Computational Methods in Astrophysics N-body Simulations and Galaxy Formation, Aún no publicado.
5. A. Burkert, The Structure of Dark Matter Halos in Dwarf Galaxies, The Astrophysical Journal 447 (1995), 10.1086/309560
6. J. Barnes and P. Hut, A hierarchical O(N log N) force-calculation algorithm, Nature 324 (1986) 446, 10.1038/324446a0